AUTOMATED SKITTLES COLOR SORTING SYSTEM

AUTHORS
CHATPOL LERDTHUWANON
JAEHYEOK KIM

COURSE
ME 4405 — FUNDAMENTALS OF MECHATRONICS

INSTITUTION
GEORGIA INSTITUTE OF TECHNOLOGY

DATE
APRIL 30, 2026

DOCUMENT
FINAL PROJECT DOCUMENTATION

Executive Summary

This document describes the design, implementation, and verification of an automated Skittles color sorting system built on the STMicroelectronics NUCLEO-L476RG development board. The system accepts a hopper of randomly mixed Skittles, indexes them one-at-a-time past a TCS34725 RGB color sensor using a stepper-driven carousel, classifies each candy into one of five colors (Red, Green, Blue, Yellow, Orange) using a normalized-RGB threshold algorithm with 5-sample majority voting, and routes the candy into the correct bin via a single SG90 servo with six pre-calibrated angular positions. The user is given live feedback through an SSD1306 OLED and five color-coded LEDs.

The final build differs from the original two-part proposal in several deliberate ways. A dedicated TCRT5000 IR beam-break sensor was eliminated after testing showed that the TCS34725's built-in clear channel could detect candy presence reliably with a tightened threshold (c > 400 AND total RGB > 250). A stepper-driven carousel (counter-clockwise indexing) replaced the second servo originally proposed for feed release, giving smoother indexing and removing one PWM channel from the system. A single SPI-driven OLED replaced the proposed array of five I^2C displays. Purple was dropped in favor of yellow + orange because the TCS34725 cleanly separates them in normalized-RGB space, while purple sits ambiguously between red and blue.

The original 90% accuracy / 180 s batch goal was preserved as a design target, but explicit batch-stop logic is not implemented in firmware — the system runs continuously after power-on and is verified by manual count over a fixed batch.

Motivation & System Objective

Motivation

Color-based sorting is a widely deployed technique in automated manufacturing inspection, food processing, pharmaceutical pill sorting, and recycling. A Skittles sorter is a compact and accessible problem that exercises the same core mechatronics building blocks used in those industrial systems: a controlled feed mechanism, an optical sensor with deterministic illumination, a microcontroller running a real-time classification algorithm, and a deterministic actuator that places parts in the correct location. Building one end-to-end demonstrates integration of sensing, actuation, embedded firmware, and electrical design within a single project.

System Objective

Design and build an automated system that accepts randomly mixed Skittles, detects the color of each Skittle, and routes it to the correct color bin without operator intervention beyond pressing power on.

Performance Goals

The performance targets carried over from Proposal Part 1 were:

- ≥ 90 of 100 randomly mixed Skittles correctly sorted (≥ 90% accuracy)
- Total processing time of 100 Skittles ≤ 180 s (average ≤ 1.8 s per Skittle)
- Skittle presence detection at the sense station
- Operator-actuated emergency stop

Implementation note

Two of these targets are met functionally but are not enforced in firmware. There is no batch counter that halts the system at N candies, and there is no emergency-stop callback wired to the user button. The performance verification method described in Section 13 therefore relies on manual stopwatch timing and a hand-counted bin tally rather than an in-firmware logger. This trade-off is discussed honestly in Section 14, Lessons Learned.

System Overview

The system is a three-stage pipeline driven by a single state loop on the NUCLEO-L476RG. Skittles flow from a hopper into a stepper-indexed carousel, are presented one at a time to the TCS34725 color sensor, classified in firmware, and finally routed by a sorting servo into one of six output bins (five colors plus an “unknown” bin at 0°). The full data path is shown below.

Block Diagram (signal flow)

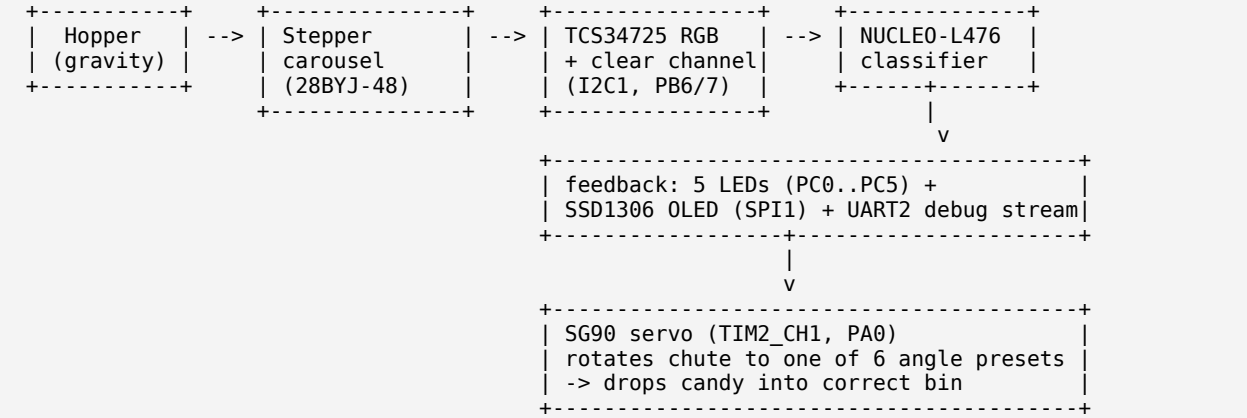


Figure 1. The assembled system sorting a live batch — gravity hopper, sensor/carousel head with the status LEDs and OLED lit, the servo-driven sorting chute, and the five color bins below.

Mechanical Design

The structure is a single vertical stack modeled in CAD: a gravity hopper feeds the stepper-indexed carousel, which presents each Skittle to the color sensor before the sorting chute routes it into one of the bins at the base. Every structural part is 3D-printed. The three load-bearing members were checked with finite element analysis before printing to confirm they carry the static and actuation loads with margin.

CAD Model

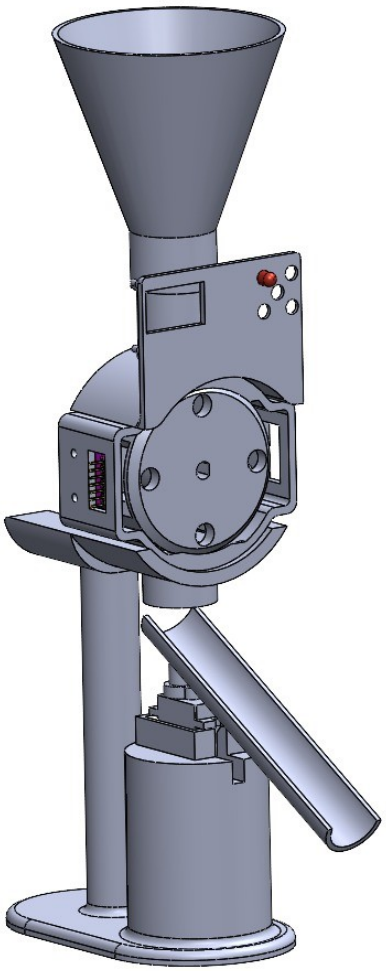


Figure 2. Full CAD assembly — gravity hopper, sensor and carousel head, sorting chute, and bin base in one vertical stack.

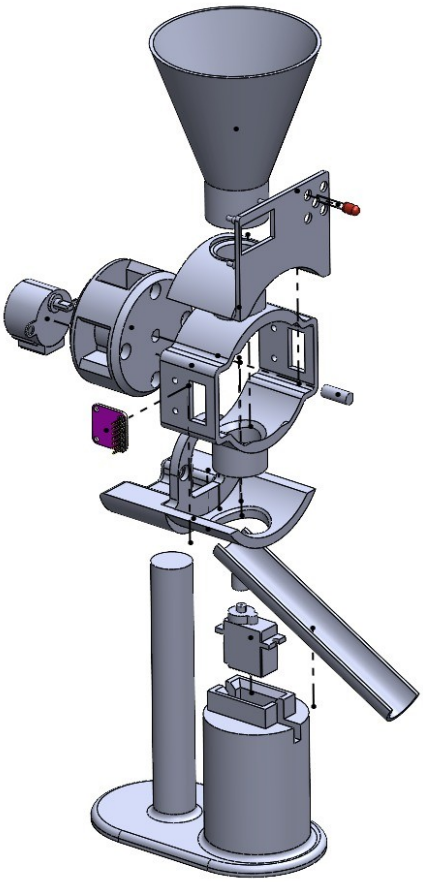
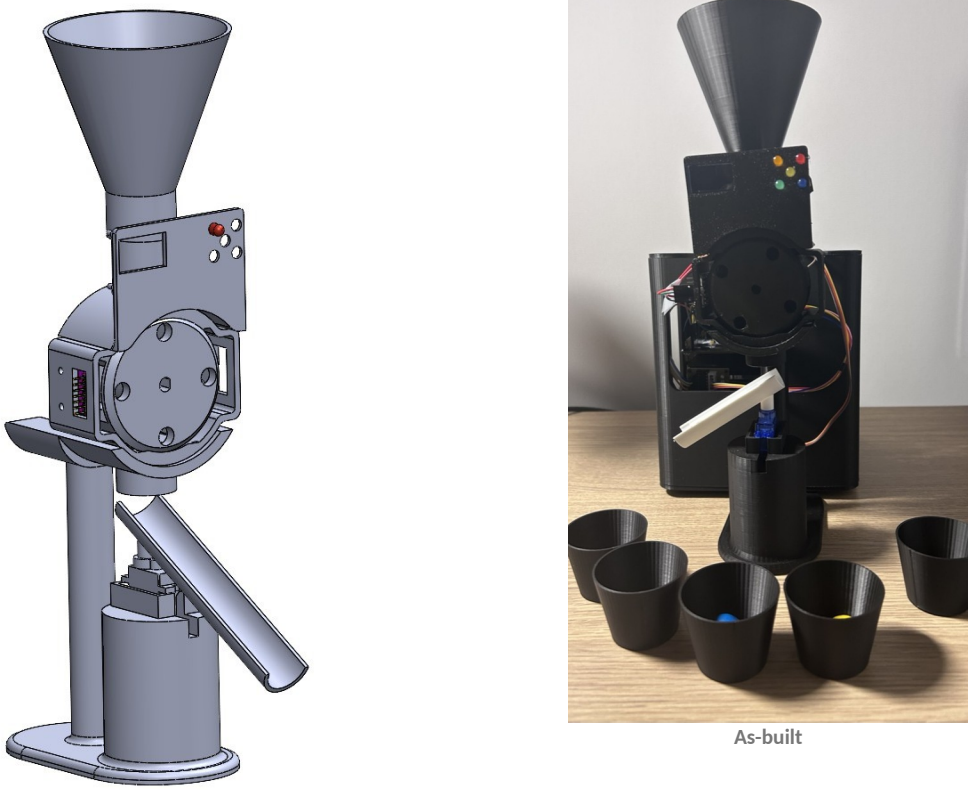


Figure 3. Exploded view — carousel disc, sensor mount, servo-driven sorting chute, and support post.

As-Built vs. Design Intent

The fabricated hardware follows the CAD model closely. Carousel pocket spacing, sensor standoff, and chute throw angles were carried straight from the model into the print, so the as-built machine matches the design intent with only the addition of wiring and electronics.



CAD model

As-built

Figure 4. Design intent (left) versus the as-built, wired hardware (right).

Finite Element Analysis

Three critical parts were analyzed under their worst-case static loads in SolidWorks Simulation. The support post was checked for tip deflection under the cantilevered weight of the sensor/carousel head; the carousel cradle and the sorting chute were checked for von Mises stress under their support and actuation loads. In every case the result sits far inside the safe range — peak stresses on the order of 0.1 MPa, orders of magnitude below the yield strength of the printed polymer — confirming the parts are structurally over-designed for the light loads in this system.

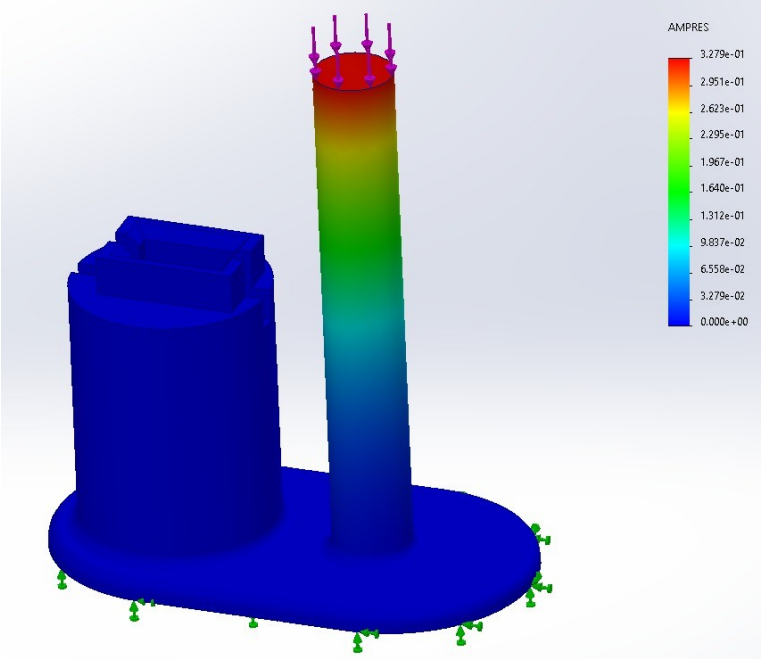


Figure 5. Support post — resultant displacement under the cantilevered head load. Peak deflection is roughly 0.33 mm at the post tip (applied loads in pink, fixed base in green).

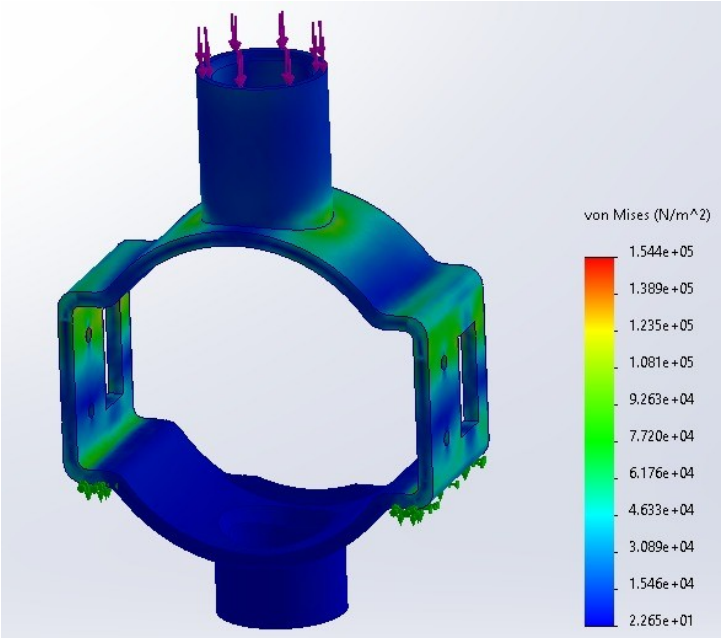


Figure 6. Carousel cradle — von Mises stress while supporting and indexing the carousel disc. Peak stress is about 0.15 MPa.

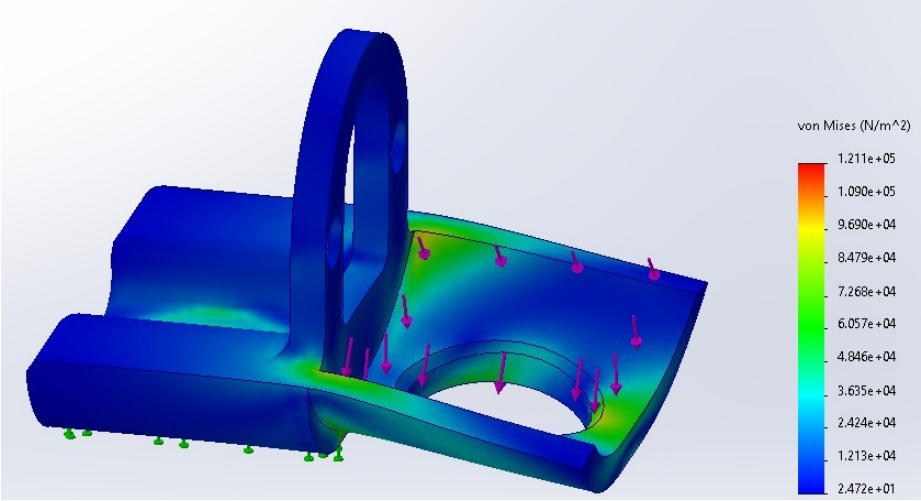


Figure 7. Sorting chute — von Mises stress under the servo actuation load as the chute swings between bin positions. Peak stress is about 0.12 MPa.

Subsystem Descriptions

6.1 Feeding Subsystem (Stepper-Driven Carousel)

- Purpose: present exactly one Skittle at a time to the optical sensor, then evacuate it after classification.
- Components: 28BYJ-48 5 V unipolar stepper, ULN2003 driver board, 3D-printed carousel disc with four evenly spaced pockets at 0°, 90°, 180°, and 270°.
- How it works: each iteration of the main loop calls Stepper_Step(STEPS_PER_90 / 4, 2, -1). The third argument is a direction flag (+1 = clockwise, anything else = counter-clockwise); the active build runs counter-clockwise. Inside Stepper_Step the four coil lines (PA8, PA9, PA4, PA6) are energised in single-phase wave-drive sequence with a 2 ms inter-pulse HAL_Delay. The outer loop runs 128 times and fires four coils per outer pass, producing 512 coil pulses per call (≈ 1024 ms wall time). After every move the firmware waits 500 ms (HAL_Delay) for mechanical settling before the sensor read, eliminating motion blur and color aliasing on the sensor face.
- All four coil lines are pulled LOW after each motion so the motor does not stay energised and overheat between cycles.

Design evolution

Proposal Part 2 specified a second SG90 “feed servo” on TIM2_CH2 (PA1) to release one candy at a time. This was replaced with a 28BYJ-48 stepper for three reasons. First, an indexing carousel naturally enforces single-Skittle delivery without the chatter that a servo flap caused in early bench tests. Second, the stepper costs less than $3, has internal gearing that makes torque a non-issue at this scale, and runs from the same 5 V rail as the rest of the system. Third, freeing TIM2_CH2 simplified the timer configuration and reduced the number of high-current rails on the board, which had previously caused servo jitter.

6.2 Color-Sensing Subsystem

- Purpose: produce a normalized R/G/B vector for each Skittle, plus detect whether a candy is actually present in the field of view.

- Components: TCS34725 RGB color sensor with onboard white LED and IR-blocking filter.
- Interface: I²C1 at 400 kHz; SCL on PB6, SDA on PB7. Sensor at 7-bit address 0x29 (write byte 0x52 after the COMMAND_BIT is OR'd in).
- Initialization: TCS34725_Init() writes ATIME = 0xD5 (~100 ms integration time), CONTROL = 0x01 (4× gain), then ENABLE = 0x03 (PON | AEN). Tuning the integration and gain explicitly gives a stable mid-range reading on every Skittle color — defaults saturated on bright candies and were noisy on dark ones.
- Read sequence: a single HAL_I2C_Mem_Read with auto-increment pulls 8 bytes (CDATA_L through BDATA_H) per cycle and assembles the 16-bit C, R, G, and B channel values.

Design evolution

Proposal Part 2 paired the TCS34725 with a TCRT5000 IR beam-break sensor on PA4 to detect Skittle presence. During integration we discovered the TCS34725's clear channel already gives a clean, monotonic signal: with no Skittle in the pocket, c stays well below 100; with a Skittle present, c climbs past 1000. A tightened gate — if (c > 400 && r+g+b > 250) — is used as the presence test, and the IR sensor was removed from the bill of materials. PA4 was repurposed as a stepper coil line.

6.3 Sorting Subsystem (Single SG90 Servo)

- Purpose: rotate the delivery chute to one of six fixed angular positions so the falling Skittle lands in the correct bin.
- Components: one SG90 9 g micro servo. Stall torque ~0.18 Nm at 4.8 V — ≥35× the calculated 0.005 Nm requirement.
- Control: PWM via TIM2_CH1 on PA0. Timer prescaler = 79 and period = 19999 give a 50 Hz frame; pulse = (angle * 2000 / 180) + 500 maps 0°–180° to 500–2500 µs.
- Six angle presets: 0° (none / pass-through), 30° (Red), 60° (Green), 90° (Blue), 120° (Yellow), 150° (Orange).

Design evolution

The proposal called for two servos — one for feed release, one for sorting. The feed servo was replaced by the stepper carousel (Section 6.1), so only one servo remains. This simplifies the power budget (one less stall-current spike on the 5 V rail) and frees TIM2_CH2 for future use.

6.4 User Feedback Subsystem

- Five status LEDs on GPIOC — PC0 (Red), PC2 (Green), PC3 (Blue), PC4 (Yellow), PC5 (Orange). One LED is lit per detection cycle to mirror the sorting servo position; all five go LOW when no Skittle is present.
- One 128×64 SSD1306 OLED on SPI1 (PA5 SCK, PA7 MOSI; RES/CS/DC on GPIOB). The display shows the most recent detected color, its individual count, and the running grand total. The display is updated only on the rising edge of a new color event to avoid flicker and unnecessary SPI traffic.
- Serial debug stream on USART2 at 115200 8N1. After every confirmed sort the firmware sends a single CSV-style line with all five counters — useful during testing on a laptop terminal.

Design evolution

Proposal Part 2 listed five OLEDs on a shared I²C bus. This was descoped to a single OLED — the count for every color fits comfortably on one 128×64 panel, and five units would have shared the I²C bus with the sensor and added five address conflicts to manage. The interface was also moved from I²C to SPI to keep the I²C1 bus dedicated to the TCS34725 (avoids any 400 kHz contention).

- LED port changed from GPIOB to GPIOC because GPIOB now hosts the SSD1306 control lines (RES, CS, DC) and the I²C1 pins. Moving the LEDs to GPIOC removes any timer-channel or alternate-function conflicts.

Color Classification Algorithm

Classification runs five times per cycle after the stepper move and the 500 ms settle. Each pass has three stages — presence gate, normalization, and confidence-ordered threshold matching — and the five sample results feed a majority vote (≥ 3 of 5 must agree) before the actuators move.

Stage 1 — Presence gate

Two conditions must be satisfied before any color decision is made:

```
if (c > 400 && (r + g + b) > 250) { /* classify */ }
```

The clear-channel threshold of 400 (paired with total-RGB > 250) was tuned empirically. Below those values the sensor is reading background or a near-empty pocket, and any RGB ratio computed from the small numerator is dominated by sensor noise.

Stage 2 — Normalization

Each channel is converted to a percentage of the RGB sum, eliminating the brightness term:

```
uint32_t total = r + g + b;
float rf = (float)r / total * 100.0f;
float gf = (float)g / total * 100.0f;
float bf = (float)b / total * 100.0f;
```

Stage 3 — Ordered threshold matching

The five color rules are evaluated in a deliberate order. Orange and Yellow are checked first because they are mixed-channel colors with both red and green high; if they were checked after the dominant-channel rules below, an Orange Skittle would mis-trigger the Red rule.

Color	Rule (using normalized rf, gf, bf)	Servo angle
Orange	rf > 40 AND 20 < gf < 35 AND bf < 20	150°
Yellow	rf > 35 AND gf > 35 AND bf < 20	120°
Red	rf > 40 AND rf > gf AND rf > bf	30°
Green	gf > 40 AND gf > rf AND gf > bf	60°
Blue	bf > 40 AND bf > rf AND bf > gf	90°
None / unknown	no rule matches (or c ≤ 400, total ≤ 250)	servo holds last position

Stage 4 — Edge-triggered counting (debounce)

A static prev_color is held across loop iterations. The counter increment and OLED refresh fire only when the voted winner differs from prev_color. A single Skittle that lingers in the pocket across multiple stepper cycles is therefore counted exactly once — acting as a software debounce on top of the per-cycle 5-of-5 voting.

Electrical Design

All 3.3 V logic peripherals (TCS34725, OLED control lines, status LEDs) are powered from the NUCLEO's onboard LD39050 regulator, which has 500 mA of headroom — a comfortable margin over the ~50–150 mA aggregate logic load. The SG90 servo and the 28BYJ-48 stepper are powered from a separate 5 V rail produced by an ELEGOO buck-converter PCB fed from a 7.4 V 2-cell LiPo, so servo stall transients and stepper inrush cannot droop the MCU rail. Grounds are tied at a single point on the ELEGOO board.

Final pinout (extracted from main.c)

The pin assignments below match the GPIO and peripheral initialisation code in main.c. Pins listed as "PBn" are tied to OLED_RES_Pin / OLED_CS_Pin / OLED_DC_Pin macros defined in main.h — they are GPIOB outputs but the specific pin numbers depend on the CubeMX configuration in main.h.

STM32 Pin	Direction	Function	Bus / Notes
PA0	Output (AF)	Sorting servo PWM	TIM2_CH1, 50 Hz, 0.5–2.5 ms
PA2	Output (AF)	USART2 TX (debug)	115200 8N1
PA3	Input (AF)	USART2 RX (debug)	115200 8N1
PA4	Output	Stepper coil IN3	ULN2003 IN3 (yellow)
PA5	Output (AF)	OLED SCK	SPI1 clock
PA6	Output	Stepper coil IN4	ULN2003 IN4 (orange)
PA7	Output (AF)	OLED MOSI / SDA	SPI1 data
PA8	Output	Stepper coil IN1	ULN2003 IN1 (blue)
PA9	Output	Stepper coil IN2	ULN2003 IN2 (pink)
PB6	Output (AF, OD)	TCS34725 SCL	I2C1 clock, 400 kHz
PB7	I/O (AF, OD)	TCS34725 SDA	I2C1 data
PBn	Output	OLED RES (reset)	Macro OLED_RES_Pin (GPIOB)
PBn	Output	OLED CS (chip select)	Macro OLED_CS_Pin (GPIOB)
PBn	Output	OLED DC (data/cmd)	Macro OLED_DC_Pin (GPIOB)
PC0	Output	Status LED — Red	330 Ω series
PC2	Output	Status LED — Green	330 Ω series

STM32 Pin	Direction	Function	Bus / Notes
PC3	Output	Status LED — Blue	330 Ω series
PC4	Output	Status LED — Yellow	330 Ω series
PC5	Output	Status LED — Orange	330 Ω series
PC13 (B1)	Input (EXTI)	User button	Falling edge — NO callback in firmware
3V3	Output	Logic rail	TCS34725, OLED, LEDs
VIN / 5V	Input	Servo + stepper rail	From ELEGOO buck → 7.4 V LiPo
GND	—	Common ground	Sensor, OLED, LEDs, motors

Power architecture

- 7.4 V LiPo → ELEGOO buck → 5 V rail → NUCLEO VIN, SG90 VCC, ULN2003 VCC.
- NUCLEO LDO → 3.3 V rail → TCS34725 VCC, SSD1306 logic, 5× LED current-limiting resistors.
- All grounds tied at the ELEGOO board single ground point.
- Estimated steady-state current draw: ~470 mA at 7.4 V (~2.14 W), giving > 2 hours of continuous runtime on a 900 mAh LiPo — well above the 3-minute demo target.

State Machine

The proposal sketched a five-state machine (IDLE → FEED → SENSE → IDENTIFY → SORT → RESET, plus an E-STOP override). The firmware actually runs as a single linear sequence inside an infinite while(1) loop, with no explicit state variable. Each loop iteration executes the same six steps:

```
while (1) {
  /* 1 */ Stepper_Step(STEPS_PER_90 / 4, 2, -1);  // index carousel 90 deg CCW
  /* 2 */ HAL_Delay(500);                          // mechanical settle
  /* 3 */ for (i=0..4) { read+classify; HAL_Delay(20); }  // 5 samples
  /* 4 */ winner = first color with >= 3/5 votes
  /* 5 */ if (winner != prev) update LED + servo + OLED
  /* 3 */ HAL_I2C_Mem_Read(... CDATA_L, 8 ...) // read C,R,G,B (16 bit each)
  /* 4 */ classify();                          // gate, normalize, threshold
  /* 5 */ if (color != prev_color) {
             update_counters(); update_oled(); uart_log();
           }
  /* 6 */ set_led(color); Set_Servo_Angle(...); // route to bin
  HAL_Delay(200);                               // inter-cycle gap
}
```

States that are not present

There is no explicit IDLE state — sorting begins immediately on power-on. There is no E-STOP state — the user button is configured as an EXTI input on falling edge, but no HAL_GPIO_EXTI_Callback handler is implemented, so a press is silently ignored. There is no RESET state — the servo simply re-commands ANGLE_NONE (0°) on the next cycle if no Skittle is detected. There is no batch-complete terminator — the loop runs forever.

Firmware Architecture

The firmware is built on STM32 HAL (CubeIDE-generated) and uses the following peripherals:

Peripheral	Role	Configuration
I2C1	TCS34725 sensor	PB6/PB7, fast mode (Timing 0x10D19CE4 → ~400 kHz)
SPI1	SSD1306 OLED	PA5/PA7, master, mode 0, prescaler 16
TIM2 CH1 PWM	Sorting servo	Prescaler 79, period 19999 → 50 Hz, 1 µs tick
USART2	Debug stream	PA2/PA3, 115200 8N1, blocking transmit
GPIOA out	Stepper coils	PA4, PA6, PA8, PA9 push-pull
GPIOC out	Status LEDs	PC0, PC2, PC3, PC4, PC5 push-pull
GPIOB out	OLED control	OLED_RES_Pin / OLED_CS_Pin / OLED_DC_Pin
EXTI	User button	B1_Pin (PC13) falling edge — callback NOT implemented

Key user functions

- TCS34725_Init(I2C_HandleTypeDef *hi2c) — writes 0x03 to ENABLE (PON | AEN).
- Set_Servo_Angle(uint16_t angle) — maps 0°–180° to 500–2500 µs and writes the timer compare register.
- Stepper_Step(int steps, uint16_t delay_ms, int direction) — wave-drive the four coil lines (direction = +1 forward, anything else reverse), with a final “all coils LOW” to prevent thermal runaway.

Main loop budget (measured / estimated)

- Stepper move (128 outer × 4 inner = 512 wave-drive pulses × 2 ms): ~1024 ms
- Settle delay: 500 ms
- I²C 8-byte burst read at 400 kHz, repeated 5×: ~5 ms total
- Classification (5 samples + vote) + display update: < 10 ms
- Inter-cycle delay: 200 ms
- Total per Skittle: ~1839 ms theoretical (≈ 33/min) — marginally over the 1.8 s proposal target; the bottleneck (Section 14) is the 1024 ms stepper move.

Calibration

The firmware uses fixed normalized-RGB thresholds rather than a learned classifier, so calibration was the process of empirically tuning those constants until the five Skittle colors separated cleanly. The procedure was:

- Place a known-color Skittle in the carousel pocket under the TCS34725 with the sensor’s onboard white LED enabled.

- Capture (c, r, g, b) for ~30 placements per color via on-the-fly USART2 reads (debug build), varying orientation slightly to mimic real flow.
- In a spreadsheet compute rf, gf, bf and inspect the per-color clusters.
- Adjust the boundaries so each color's rule excludes the other four with margin to spare — the values shipped in main.c (40 % / 35 % / 20 % breakpoints) are the result.
- Cross-check c readings with empty pockets and small candies to set the presence gate at c > 400 with total > 250.

Because the TCS34725 has an integrated IR-blocking filter and an on-package white LED, ambient light contamination is small. The same threshold set has held up across two ambient lighting conditions on the lab bench (overhead fluorescent and direct desk LED) without re-tuning. A formal classifier (k-NN, decision tree, or a 5-class SVM on rf/gf/bf) was considered but unnecessary given the clean separation observed.

Bill of Materials

Final BOM. Items in italics were proposed but not used in the build.

Component	Qty	Unit cost	Status
NUCLEO-L476RG dev board	1	$29.24	Used
TCS34725 RGB sensor module	1	$13.99	Used
28BYJ-48 stepper + ULN2003 driver	1	$3.50	Used (replaces feed servo)
SG90 micro servo	1	$2.00	Used (sorting only)
SSD1306 128×64 OLED (SPI, 7-pin)	1	$5.99	Used
5 mm LEDs (R/G/B/Y/O)	5	$0.10	Used
330 Ω resistors	5	$0.05	Used (LED current limit)
7.4 V 2-cell LiPo (ELEGOO Tumbler kit)	1	—	Used
ELEGOO buck-converter PCB	1	—	Used
3D-printed carousel + chute + bins	1 set	—	Used
TCRT5000 IR beam-break sensor	1	$9.32	Originally proposed, not used
Additional SSD1306 OLEDs (4)	4	$5.99	Originally proposed, not used
Second SG90 servo (feed release)	1	$2.00	Originally proposed, not used
Emergency-stop momentary push-button	1	$0.10	Originally proposed, not wired in firmware
Start momentary push-button	1	$0.10	Originally proposed, not wired in firmware

Testing & Results

Methodology

- Pre-test: weigh out a fixed batch of Skittles (target N = 100) with a verified count of each color.
- Power on the system. Start a stopwatch when the first stepper move begins.
- Allow the system to run continuously until the hopper is empty. Stop the stopwatch when the OLED count stops incrementing on the final candy.
- Manually count the candies in each output bin and tally misses (correct color in wrong bin or unknown bin).
- Compute accuracy = correctly-binned / total, and average time = total time / total.

Measured results

Test data must be filled in after the verification run. The placeholders below are intentionally not invented.

Metric	Target	Measured
Accuracy (correctly binned / total)	≥ 90 %	[INSERT: measured accuracy %]
Total time for N Skittles	≤ 180 s (for N=100)	[INSERT: measured time]
Average time per Skittle	≤ 1.8 s	[INSERT: avg s/Skittle]
Per-color accuracy (Red)	n/a	[INSERT: %]
Per-color accuracy (Green)	n/a	[INSERT: %]
Per-color accuracy (Blue)	n/a	[INSERT: %]
Per-color accuracy (Yellow)	n/a	[INSERT: %]
Per-color accuracy (Orange)	n/a	[INSERT: %]
Confused pairs (most-frequent miss)	n/a	[INSERT: e.g. orange → yellow]

Lessons Learned & Design Trade-offs

What worked

- Reusing the TCS34725 clear channel as the presence detector — one sensor, two jobs, fewer parts.
- Stepper-driven carousel for feeding — deterministic 90 degree indexing without closed-loop feedback.
- Normalized-RGB threshold classifier — ~30 lines, five colors cleanly separated, no ML.

What was descoped, and why

- IR beam-break sensor and second servo — replaced by the clear channel and the stepper carousel.
- Five OLEDs to one OLED on SPI — all counts fit on 128x64 and SPI keeps I2C1 dedicated to the sensor.
- E-stop button and batch-complete terminator — not wired in firmware; verification done by manual stopwatch and bin tally.

What we would improve next iteration

- Implement HAL_GPIO_EXTI_Callback so the user button gives a real start, pause, and E-stop.
- Add a batch counter that halts the loop and prints final statistics when count_total hits a configurable N.
- Replace blocking HAL_Delay calls with a non-blocking scheduler and switch to a faster stepper drive so the cycle time drops well below the current ~1.84 s.

Appendix A — Full Pinout

STM32 Pin	Direction	Function	Bus / Notes
PA0	Output (AF)	Sorting servo PWM	TIM2_CH1, 50 Hz, 0.5–2.5 ms
PA2	Output (AF)	USART2 TX (debug)	115200 8N1
PA3	Input (AF)	USART2 RX (debug)	115200 8N1
PA4	Output	Stepper coil IN3	ULN2003 IN3 (yellow)
PA5	Output (AF)	OLED SCK	SPI1 clock
PA6	Output	Stepper coil IN4	ULN2003 IN4 (orange)
PA7	Output (AF)	OLED MOSI / SDA	SPI1 data
PA8	Output	Stepper coil IN1	ULN2003 IN1 (blue)
PA9	Output	Stepper coil IN2	ULN2003 IN2 (pink)
PB6	Output (AF, OD)	TCS34725 SCL	I2C1 clock, 400 kHz
PB7	I/O (AF, OD)	TCS34725 SDA	I2C1 data
PBn	Output	OLED RES (reset)	Macro OLED_RES_Pin (GPIOB)
PBn	Output	OLED CS (chip select)	Macro OLED_CS_Pin (GPIOB)
PBn	Output	OLED DC (data/cmd)	Macro OLED_DC_Pin (GPIOB)
PC0	Output	Status LED — Red	330 Ω series
PC2	Output	Status LED — Green	330 Ω series
PC3	Output	Status LED — Blue	330 Ω series
PC4	Output	Status LED — Yellow	330 Ω series
PC5	Output	Status LED — Orange	330 Ω series
PC13 (B1)	Input (EXTI)	User button	Falling edge — NO callback in firmware
3V3	Output	Logic rail	TCS34725, OLED, LEDs
VIN / 5V	Input	Servo + stepper rail	From ELEGOO buck → 7.4 V LiPo
GND	—	Common ground	Sensor, OLED, LEDs, motors

Source Code

The full firmware source ships with the project at Core/Src/main.c (716 lines). The relevant pinout, peripheral configuration, classification thresholds, and main-loop sequence are reproduced verbatim

throughout this document; the remaining lines are CubeMX-generated boilerplate (clock setup, MX_*_Init, Error_Handler, assert_failed) and are not reproduced here.